

00_prepare_dataset

September 27, 2026

1 00 — Prepare Dataset

This notebook migrates the old Python data-preparation workflow into a reproducible notebook and protects the frozen dataset from accidental changes.

It prepares/verifies:

```
data/  
  core_single_4species_metadata.csv  
  core_single_4species_frozen_split.csv  
  core_audio_audit.csv
```

Safety rule: **reproduce first, compare against the existing frozen artifacts, and stop on any mismatch**. The canonical CSVs are not overwritten by default.

Legacy Core: - sound_type == "mosquito" - plurality == "Single" - native sample_rate == 44100 - an arabiensis, an funestus ss, an squamosus, culex pipiens complex

Legacy split design: group by original recording name, split per species, deterministic seed 42, source ratios 70% train / 15% validation / remainder test, **before segmentation**.

```
[1]: from pathlib import Path  
import wave, json, hashlib  
import numpy as np  
import pandas as pd  
  
PROJECT = next((p for p in [Path.cwd(), *Path.cwd().parents]  
                if (p / "data/neurips_2021_zenodo_0_0_1.csv").is_file()), None)  
assert PROJECT is not None, "Run from the project root or its notebook_↵  
  ↵directory"  
DATA_DIR = PROJECT / "data"  
AUDIO_DIR = DATA_DIR / "audio"  
  
RAW_METADATA_PATH = DATA_DIR / "neurips_2021_zenodo_0_0_1.csv"  
CORE_PATH = DATA_DIR / "core_single_4species_metadata.csv"  
SPLIT_PATH = DATA_DIR / "core_single_4species_frozen_split.csv"  
AUDIO_AUDIT_PATH = DATA_DIR / "core_audio_audit.csv"  
  
REPRO_DIR = DATA_DIR / "_reproduced_00"  
REPRO_DIR.mkdir(parents=True, exist_ok=True)
```

```

for p in [RAW_METADATA_PATH, CORE_PATH, SPLIT_PATH, AUDIO_AUDIT_PATH]:
    assert p.exists(), f"Missing required file: {p}"
assert AUDIO_DIR.exists(), f"Missing audio directory: {AUDIO_DIR}"

print("All required inputs found.")
CANONICAL_START_HASHES = {p: hashlib.sha256(p.read_bytes()).hexdigest()
                          for p in [CORE_PATH, SPLIT_PATH, AUDIO_AUDIT_PATH]}

```

All required inputs found.

1.1 00.1 — Verify raw metadata

```

[2]: metadata = pd.read_csv(RAW_METADATA_PATH)
print("Shape:", metadata.shape)
print("Unique IDs:", metadata["id"].nunique())
print("Duplicate IDs:", metadata["id"].duplicated().sum())

assert metadata.shape == (9295, 19)
assert metadata["id"].is_unique
print("Raw metadata: PASS")

```

Shape: (9295, 19)
Unique IDs: 9295
Duplicate IDs: 0
Raw metadata: PASS

1.2 00.2 — Recreate the strict four-species Core

```

[3]: CORE_SPECIES = [
    "an arabiensis",
    "an funestus ss",
    "an squamosus",
    "culex pipiens complex",
]

reproduced_core = metadata[
    (metadata["sound_type"] == "mosquito")
    & (metadata["plurality"] == "Single")
    & (metadata["sample_rate"] == 44100)
    & (metadata["species"].isin(CORE_SPECIES))
].copy()

print("Core clips:", len(reproduced_core))
print("Core sources:", reproduced_core["name"].nunique())
print("\nClips/species:")
print(reproduced_core["species"].value_counts())
print("\nSources/species:")

```

```

print(reproduced_core.groupby("species")["name"].nunique())

assert len(reproduced_core) == 1898
assert reproduced_core["id"].nunique() == 1898
assert reproduced_core["name"].nunique() == 1189
assert reproduced_core["species"].value_counts().to_dict() == {
    "an arabiensis": 831,
    "culex pipiens complex": 545,
    "an funestus ss": 381,
    "an squamosus": 141,
}
assert reproduced_core.groupby("species")["name"].nunique().to_dict() == {
    "an arabiensis": 514,
    "an funestus ss": 248,
    "an squamosus": 91,
    "culex pipiens complex": 336,
}
print("Core counts: PASS")

```

Core clips: 1898

Core sources: 1189

Clips/species:

```

species
an arabiensis      831
culex pipiens complex  545
an funestus ss      381
an squamosus       141
Name: count, dtype: int64

```

Sources/species:

```

species
an arabiensis      514
an funestus ss      248
an squamosus        91
culex pipiens complex  336
Name: name, dtype: int64
Core counts: PASS

```

```

[4]: frozen_core = pd.read_csv(CORE_PATH)
assert set(reproduced_core["id"]) == set(frozen_core["id"])

critical = ["id", "name", "species", "sample_rate", "sound_type", "plurality"]
a = reproduced_core[critical].sort_values("id").reset_index(drop=True)
b = frozen_core[critical].sort_values("id").reset_index(drop=True)
assert a.equals(b), "Critical Core metadata differs from the frozen artifact"

```

```

la = reproduced_core.sort_values("id")["length"].to_numpy(float)
lb = frozen_core.sort_values("id")["length"].to_numpy(float)
assert np.allclose(la, lb, rtol=0, atol=1e-12)
print("Exact frozen Core comparison: PASS")

```

Exact frozen Core comparison: PASS

1.3 00.3 — Reproduce the legacy source-safe split

Exact deterministic assignments depend on implementation details such as source ordering and RNG lifetime. Instead of guessing, this notebook tests legacy-compatible variants and accepts one **only if every id → split mapping equals the already-frozen split.**

```

[5]: SEED = 42
TRAIN_FRAC = 0.70
VAL_FRAC = 0.15

def species_orders(core):
    opts = [
        ("core_species_list", CORE_SPECIES),
        ("sorted_species", sorted(core["species"].unique())),
        ("encounter_species", list(pd.unique(core["species"]))),
    ]
    out, seen = [], set()
    for label, order in opts:
        t = tuple(order)
        if t not in seen:
            out.append((label, list(order)))
            seen.add(t)
    return out

def source_names(df, mode):
    if mode == "encounter":
        return np.array(pd.unique(df["name"]), dtype=object)
    if mode == "sorted":
        return np.array(sorted(df["name"].unique()), dtype=object)
    raise ValueError(mode)

def make_split(core, sp_order, source_order_mode, rng_mode):
    assignments = {}
    global_rng = np.random.default_rng(SEED) if rng_mode == "global" else None

    for sp in sp_order:
        sdf = core[core["species"] == sp]
        src = source_names(sdf, source_order_mode).copy()

```

```

    rng = global_rng if rng_mode == "global" else np.random.
↪default_rng(SEED)
    rng.shuffle(src)

    n_train = int(np.floor(TRAIN_FRAC * len(src)))
    n_val = int(np.floor(VAL_FRAC * len(src)))

    for s in src[:n_train]: assignments[s] = "train"
    for s in src[n_train:n_train+n_val]: assignments[s] = "validation"
    for s in src[n_train+n_val:]: assignments[s] = "test"

    out = core.copy()
    out["split"] = out["name"].map(assignments)
    assert out["split"].notna().all()
    assert out.groupby("name")["split"].nunique().max() == 1
    return out

```

```

[6]: frozen_split = pd.read_csv(SPLIT_PATH)
    assert len(frozen_split) == 1898
    assert frozen_split["id"].is_unique
    assert set(frozen_split["id"]) == set(reproduced_core["id"])

    truth = frozen_split[["id", "split"]].sort_values("id").reset_index(drop=True)
    results = []

    for sp_label, sp_order in species_orders(reproduced_core):
        for src_mode in ["encounter", "sorted"]:
            for rng_mode in ["global", "per_species"]:
                cand = make_split(reproduced_core, sp_order, src_mode, rng_mode)
                cm = cand[["id", "split"]].sort_values("id").reset_index(drop=True)
                mismatch = int((cm["split"].to_numpy() != truth["split"].
↪to_numpy()).sum())
                results.append({
                    "species_order_label": sp_label,
                    "species_order": tuple(sp_order),
                    "source_order": src_mode,
                    "rng_mode": rng_mode,
                    "mismatched_ids": mismatch,
                    "exact_match": mismatch == 0,
                })

    recipe_results = pd.DataFrame(results).
    ↪sort_values(["exact_match", "mismatched_ids"], ascending=[False, True])
    display(recipe_results)
    exact = recipe_results[recipe_results["exact_match"]]
    print("Exact recipes found:", len(exact))

```

```

assert len(exact) >= 1, "No tested legacy recipe exactly reproduces the frozen_
↳split. STOP."
print("Legacy split reproduction search: PASS")

```

	species_order_label	species_order \
0	core_species_list	(an arabiensis, an funestus ss, an squamosus, ...
2	core_species_list	(an arabiensis, an funestus ss, an squamosus, ...
1	core_species_list	(an arabiensis, an funestus ss, an squamosus, ...
3	core_species_list	(an arabiensis, an funestus ss, an squamosus, ...
5	encounter_species	(an arabiensis, culex pipiens complex, an fune...
7	encounter_species	(an arabiensis, culex pipiens complex, an fune...
4	encounter_species	(an arabiensis, culex pipiens complex, an fune...
6	encounter_species	(an arabiensis, culex pipiens complex, an fune...

	source_order	rng_mode	mismatched_ids	exact_match
0	encounter	global	0	True
2	sorted	global	0	True
1	encounter	per_species	489	False
3	sorted	per_species	489	False
5	encounter	per_species	489	False
7	sorted	per_species	489	False
4	encounter	global	493	False
6	sorted	global	493	False

Exact recipes found: 2

Legacy split reproduction search: PASS

```

[7]: chosen = exact.iloc[0]
chosen_order = list(chosen["species_order"])
reproduced_split = make_split(
    reproduced_core,
    chosen_order,
    chosen["source_order"],
    chosen["rng_mode"],
)

rid = reproduced_split[["id", "split"]].sort_values("id").reset_index(drop=True)
fid = frozen_split[["id", "split"]].sort_values("id").reset_index(drop=True)
assert rid.equals(fid)

rname = reproduced_split[["name", "split"]].drop_duplicates().
↳sort_values("name").reset_index(drop=True)
fname = frozen_split[["name", "split"]].drop_duplicates().sort_values("name").
↳reset_index(drop=True)
assert rname.equals(fname)

assert reproduced_split["split"].value_counts().to_dict() == {
    "train": 1335, "test": 294, "validation": 269
}

```

```

}
assert reproduced_split.groupby("split")["name"].nunique().to_dict() == {
    "test": 182, "train": 830, "validation": 177
}
assert reproduced_split.groupby("name")["split"].nunique().max() == 1

print("Chosen exact legacy recipe:")
print(" species order:", chosen_order)
print(" source order:", chosen["source_order"])
print(" RNG mode:", chosen["rng_mode"])
print(" seed:", SEED)
print("Exact id -> split mapping: PASS")
print("Exact name -> split mapping: PASS")
print("Source leakage: 0")

```

```

Chosen exact legacy recipe:
 species order: ['an arabiensis', 'an funestus ss', 'an squamosus', 'culex
 pipiens complex']
 source order: encounter
 RNG mode: global
 seed: 42
Exact id -> split mapping: PASS
Exact name -> split mapping: PASS
Source leakage: 0

```

1.4 00.4 — Acquisition/domain audit (diagnostic only)

```

[8]: for col in
    ↪ ["device_type", "mic_type", "method", "country", "place", "record_datetime"]:
    if col in reproduced_split.columns:
        print("\n" + "="*70)
        print(col.upper())
        print("="*70)
        display(pd.crosstab(
            [reproduced_split["species"], reproduced_split["split"]],
            reproduced_split[col].fillna("<NA>")
        ))

```

```

=====
DEVICE_TYPE
=====

device_type          tascam
species              split
an arabiensis        test      124
                   train      588
                   validation  119

```

an funestus ss	test	61
	train	267
	validation	53
an squamosus	test	23
	train	99
	validation	19
culex pipiens complex	test	86
	train	381
	validation	78

MIC_TYPE

mic_type	telinga	
species	split	
an arabiensis	test	124
	train	588
	validation	119
an funestus ss	test	61
	train	267
	validation	53
an squamosus	test	23
	train	99
	validation	19
culex pipiens complex	test	86
	train	381
	validation	78

METHOD

method		HBN	LC	LT
species	split			
an arabiensis	test	59	0	65
	train	201	0	387
	validation	43	0	76
an funestus ss	test	12	0	49
	train	57	0	210
	validation	7	0	46
an squamosus	test	0	0	23
	train	0	0	99
	validation	0	0	19
culex pipiens complex	test	43	0	43
	train	203	3	175
	validation	44	0	34

=====

COUNTRY

=====

country		Tanzania
species	split	
an arabiensis	test	124
	train	588
	validation	119
an funestus ss	test	61
	train	267
	validation	53
an squamosus	test	23
	train	99
	validation	19
culex pipiens complex	test	86
	train	381
	validation	78

=====

PLACE

=====

place		Ifakara
species	split	
an arabiensis	test	124
	train	588
	validation	119
an funestus ss	test	61
	train	267
	validation	53
an squamosus	test	23
	train	99
	validation	19
culex pipiens complex	test	86
	train	381
	validation	78

=====

RECORD_DATETIME

=====

record_datetime		02-03-20 00:00	02-08-20 00:00	\
species	split			
an arabiensis	test	6	5	
	train	26	35	
	validation	12	10	

an funestus ss	test	0	1
	train	0	5
	validation	0	0
an squamosus	test	0	0
	train	0	0
	validation	0	0
culex pipiens complex	test	2	5
	train	23	11
	validation	2	1
record_datetime		03-02-20 00:00	04-03-20 00:00 \
species	split		
an arabiensis	test	16	4
	train	25	21
	validation	8	4
an funestus ss	test	3	5
	train	17	16
	validation	0	2
an squamosus	test	0	0
	train	0	0
	validation	0	0
culex pipiens complex	test	0	2
	train	1	13
	validation	0	0
record_datetime		05-07-20 00:00	06-07-20 00:00 \
species	split		
an arabiensis	test	5	3
	train	13	6
	validation	0	0
an funestus ss	test	0	0
	train	0	2
	validation	0	0
an squamosus	test	0	0
	train	3	5
	validation	0	0
culex pipiens complex	test	0	0
	train	7	1
	validation	4	0
record_datetime		07-03-20 00:00	07-07-20 00:00 \
species	split		
an arabiensis	test	5	10
	train	10	35
	validation	1	8
an funestus ss	test	1	5
	train	0	17
	validation	0	2

an squamosus	test	0	0
	train	0	1
	validation	0	0
culex pipiens complex	test	2	2
	train	7	8
	validation	0	0
record_datetime		08-02-20 00:00	10-02-20 00:00 ... \
species	split		...
an arabiensis	test	11	3 ...
	train	20	15 ...
	validation	2	0 ...
an funestus ss	test	0	0 ...
	train	0	0 ...
	validation	0	0 ...
an squamosus	test	0	0 ...
	train	0	0 ...
	validation	0	0 ...
culex pipiens complex	test	0	6 ...
	train	2	23 ...
	validation	0	13 ...
record_datetime		26-03-20 00:00	26-07-20 00:00 \
species	split		
an arabiensis	test	2	0
	train	10	7
	validation	3	2
an funestus ss	test	4	2
	train	0	5
	validation	0	0
an squamosus	test	0	0
	train	1	0
	validation	2	0
culex pipiens complex	test	0	0
	train	3	0
	validation	0	0
record_datetime		27-03-20 00:00	27-07-20 00:00 \
species	split		
an arabiensis	test	0	3
	train	0	4
	validation	2	2
an funestus ss	test	0	2
	train	0	5
	validation	0	0
an squamosus	test	1	0
	train	13	0
	validation	1	0

culex pipiens complex	test	0	0
	train	0	3
	validation	0	1
record_datetime		28-06-20 00:00	28-07-20 00:00 \
species	split		
an arabiensis	test	1	7
	train	4	40
	validation	3	6
an funestus ss	test	0	3
	train	0	14
	validation	0	3
an squamosus	test	2	0
	train	14	0
	validation	2	0
culex pipiens complex	test	2	0
	train	20	6
	validation	1	2
record_datetime		29-02-20 00:00	29-06-20 00:00 \
species	split		
an arabiensis	test	0	0
	train	6	4
	validation	6	0
an funestus ss	test	0	0
	train	2	1
	validation	0	0
an squamosus	test	0	1
	train	0	13
	validation	0	2
culex pipiens complex	test	4	2
	train	10	16
	validation	3	8
record_datetime		30-01-20 00:00	30-06-20 00:00
species	split		
an arabiensis	test	1	0
	train	2	1
	validation	0	0
an funestus ss	test	0	0
	train	0	0
	validation	0	0
an squamosus	test	0	6
	train	0	8
	validation	0	2
culex pipiens complex	test	5	7
	train	17	19
	validation	9	1

[12 rows x 41 columns]

1.5 00.5 — Recreate and verify the WAV audit

```
[9]: rows=[]
for n, (_, row) in enumerate(reproduced_split.iterrows(), start=1):
    cid=int(row["id"])
    p=AUDIO_DIR / f"{cid}.wav"
    if not p.exists():
        rows.append({"id":cid,"exists":False})
        continue
    with wave.open(str(p), "rb") as wf:
        sr=wf.getframerate(); ch=wf.getnchannels(); sw=wf.getsampwidth();
        frames=wf.getnframes()
        dur=frames/sr
        rows.append({
            "id":cid,
            "exists":True,
            "sample_rate_wav":sr,
            "channels":ch,
            "sample_width_bytes":sw,
            "frames":frames,
            "duration_wav":dur,
            "length":row["length"],
            "duration_diff_s":abs(dur-row["length"]),
        })
    if n % 250 == 0 or n == len(reproduced_split):
        print(f"Audited {n}/{len(reproduced_split)}")

reproduced_audio_audit=pd.DataFrame(rows)
assert reproduced_audio_audit["exists"].all()
assert (reproduced_audio_audit["sample_rate_wav"]==44100).all()
assert (reproduced_audio_audit["channels"]==1).all()
assert (reproduced_audio_audit["sample_width_bytes"]==3).all()
assert (reproduced_audio_audit["duration_diff_s"]<=0.010).all()
print("Recreated WAV audit: PASS")
```

```
Audited 250/1898
Audited 500/1898
Audited 750/1898
Audited 1000/1898
Audited 1250/1898
Audited 1500/1898
Audited 1750/1898
Audited 1898/1898
Recreated WAV audit: PASS
```

```

[10]: frozen_audit = pd.read_csv(AUDIO_AUDIT_PATH).rename(
    columns={
        "sample_rate": "sample_rate_wav",
        "duration_metadata": "length",
        "duration_difference": "duration_diff_s",
    }
)

assert set(reproduced_audio_audit["id"]) == set(frozen_audit["id"])

ra = reproduced_audio_audit.sort_values("id").reset_index(drop=True)
fa = frozen_audit.sort_values("id").reset_index(drop=True)

for col in ["id", "sample_rate_wav", "channels", "sample_width_bytes",
    ↪ "frames"]:
    assert np.array_equal(
        ra[col].to_numpy(),
        fa[col].to_numpy()
    ), f"Audit mismatch: {col}"

for col in ["duration_wav", "length", "duration_diff_s"]:
    assert np.allclose(
        ra[col].to_numpy(float),
        fa[col].to_numpy(float),
        rtol=0,
        atol=1e-10,
        equal_nan=True,
    ), f"Audit mismatch: {col}"

print("Frozen audio-audit comparison: PASS")

```

Frozen audio-audit comparison: PASS

1.6 00.6 — Save verified reproduced copies

```

[11]: REPRO_CORE = REPRO_DIR / "core_single_4species_metadata.csv"
REPRO_SPLIT = REPRO_DIR / "core_single_4species_frozen_split.csv"
REPRO_AUDIT = REPRO_DIR / "core_audio_audit.csv"
REPRO_RECIPE = REPRO_DIR / "legacy_split_recipe.json"

reproduced_core.to_csv(REPRO_CORE, index=False)
reproduced_split.to_csv(REPRO_SPLIT, index=False)
reproduced_audio_audit.to_csv(REPRO_AUDIT, index=False)

recipe_record={
    "seed": SEED,
    "train_fraction_sources_per_species": TRAIN_FRAC,
}

```

```

        "validation_fraction_sources_per_species": VAL_FRAC,
        "test_fraction": "remainder",
        "species_order": chosen_order,
        "source_order_mode": chosen["source_order"],
        "rng_mode": chosen["rng_mode"],
        "group_column": "name",
        "split_before_segmentation": True,
    }
    REPRO_RECIPES.write_text(json.dumps(recipe_record, indent=2), encoding="utf-8")

    print("Saved verified copies to:", REPRO_DIR)

```

Saved verified copies to: d:\VGU-27\Edge AI Project\data_reproduced_00

1.7 00.7 — Full semantic comparison before optional promotion

```

[12]: import io

def verified_promotion_payloads():
    """Re-read candidates and validate ALL columns before any canonical write.
    ↪"""
    old_core = pd.read_csv(CORE_PATH)
    old_split = pd.read_csv(SPLIT_PATH)
    old_audit = pd.read_csv(AUDIO_AUDIT_PATH)
    new_core = pd.read_csv(REPRO_CORE)
    new_split = pd.read_csv(REPRO_SPLIT)
    new_audit = pd.read_csv(REPRO_AUDIT).rename(columns={
        "sample_rate_wav": "sample_rate", "length": "duration_metadata",
        "duration_diff_s": "duration_difference",
    })
    # Preserve the canonical audit schema, including species and split.
    if "species" not in new_audit.columns and "split" not in new_audit.columns:
        new_audit = new_audit.merge(new_split[["id", "species", "split"]],
                                    on="id", how="left", validate="one_to_one")
    assert set(new_audit.columns) == set(old_audit.columns), "Unexpected audit_
    ↪schema"
    new_audit = new_audit.loc[:, old_audit.columns]
    payloads = {}
    for path, old, new, atol in [
        (CORE_PATH, old_core, new_core, 1e-12),
        (SPLIT_PATH, old_split, new_split, 1e-12),
        (AUDIO_AUDIT_PATH, old_audit, new_audit, 1e-10),
    ]:
        assert hashlib.sha256(path.read_bytes()).hexdigest() ==_
        ↪CANONICAL_START_HASHES[path], f"Canonical input changed_
        ↪during this run: {path}"
        assert old["id"].is_unique and new["id"].is_unique

```

```

    assert list(old.columns) == list(new.columns), f"Schema differs: {path.
↪name}"
    pd.testing.assert_frame_equal(
        old.sort_values("id").reset_index(drop=True),
        new.sort_values("id").reset_index(drop=True),
        check_dtype=False, check_exact=False, rtol=0, atol=atol)
    # Match canonical row order and validate the serialized promotion ↪
↪candidate too.
    new = new.set_index("id").loc[old["id"]].reset_index()[old.columns]
    payload = new.to_csv(index=False).encode("utf-8")
    roundtrip = pd.read_csv(io.BytesIO(payload))
    pd.testing.assert_frame_equal(old, roundtrip, check_dtype=False,
                                check_exact=False, rtol=0, atol=atol)

    payloads[path] = payload
    for keys in [{"id", "split"}, {"name", "split"}]:
        pd.testing.assert_frame_equal(
            old_split[keys].drop_duplicates().sort_values(keys).
↪reset_index(drop=True),
            new_split[keys].drop_duplicates().sort_values(keys).
↪reset_index(drop=True),
            check_exact=True)
    return payloads

verified_payloads = verified_promotion_payloads()
print("Full Core, split, and audio-audit semantic comparisons: PASS")
print("Exact id/name-to-split mappings and canonical-schema round trips: PASS")

```

Full Core, split, and audio-audit semantic comparisons: PASS

Exact id/name-to-split mappings and canonical-schema round trips: PASS

1.8 00.8 — Final dataset gate

```

[13]: summary=pd.Series({
    "raw_metadata_rows":len(metadata),
    "core_clips":len(reproduced_core),
    "core_sources":reproduced_core["name"].nunique(),
    "train_clips":int((reproduced_split["split"]=="train").sum()),
    "validation_clips":int((reproduced_split["split"]=="validation").sum()),
    "test_clips":int((reproduced_split["split"]=="test").sum()),
    "train_sources":reproduced_split.
↪loc[reproduced_split["split"]=="train","name"].nunique(),
    "validation_sources":reproduced_split.
↪loc[reproduced_split["split"]=="validation","name"].nunique(),
    "test_sources":reproduced_split.
↪loc[reproduced_split["split"]=="test","name"].nunique(),
    "source_leakage":reproduced_split.groupby("name")["split"].nunique().gt(1).
↪sum(),

```

```

    "missing_wav":int((~reproduced_audio_audit["exists"]).sum()),
    "non_44100_hz":int((reproduced_audio_audit["sample_rate_wav"]!=44100).
↪sum()),
    "non_mono":int((reproduced_audio_audit["channels"]!=1).sum()),
    "non_pcm24":int((reproduced_audio_audit["sample_width_bytes"]!=3).sum()),
    "duration_mismatch_gt_10ms":
↪int((reproduced_audio_audit["duration_diff_s"]>0.010).sum()),
    "exact_matching_split_recipes":len(exact),
})
display(summary.to_frame("value"))
print("\nDetected exact legacy recipe:")
print(json.dumps(recipe_record, indent=2))
print("\n00_PREPARE_DATASET: PASS")

```

	value
raw_metadata_rows	9295
core_clips	1898
core_sources	1189
train_clips	1335
validation_clips	269
test_clips	294
train_sources	830
validation_sources	177
test_sources	182
source_leakage	0
missing_wav	0
non_44100_hz	0
non_mono	0
non_pcm24	0
duration_mismatch_gt_10ms	0
exact_matching_split_recipes	2

Detected exact legacy recipe:

```

{
  "seed": 42,
  "train_fraction_sources_per_species": 0.7,
  "validation_fraction_sources_per_species": 0.15,
  "test_fraction": "remainder",
  "species_order": [
    "an arabiensis",
    "an funestus ss",
    "an squamosus",
    "culex pipiens complex"
  ],
  "source_order_mode": "encounter",
  "rng_mode": "global",
  "group_column": "name",

```

```

    "split_before_segmentation": true
}

```

00_PREPARE_DATASET: PASS

1.9 00.9 — Optional canonical promotion (disabled by default)

All full comparisons and assertions above must pass first. The promotion cell re-reads and revalidates candidates. It preserves every canonical column, including audio-audit species/split. Ordinary verification must leave `WRITE_CANONICAL = False`.

```

[14]: # DANGER: intentional promotion only; never enable for an ordinary audit.
WRITE_CANONICAL = False

# Revalidate here: a stale notebook flag or out-of-order execution is
↳insufficient.
verified_payloads = verified_promotion_payloads()
if WRITE_CANONICAL:
    for path, payload in verified_payloads.items():
        assert hashlib.sha256(path.read_bytes()).hexdigest() ==
↳CANONICAL_START_HASHES[path]
        path.write_bytes(payload)
        print("Verified candidates promoted with canonical schemas preserved.")
else:
    assert all(hashlib.sha256(p.read_bytes()).hexdigest() == h
               for p, h in CANONICAL_START_HASHES.items())
    print("Canonical CSV bytes unchanged. 00_PREPARE_DATASET: PASS")

```

Canonical CSV bytes unchanged. 00_PREPARE_DATASET: PASS

After this notebook passes:

```

00_prepare_dataset.ipynb
    ↓ exact reproduction verified
01_dataset_audit.ipynb
    ↓ independent audit
02_segmentation.ipynb

```

00 is the build/reproduction layer; 01 remains the independent audit layer.

```

[17]: from pathlib import Path
import subprocess
import sys

NOTEBOOK_PATH = PROJECT / "notebooks" / "current" / "00_prepare_dataset.ipynb"
PDF_DIR = PROJECT / "reports" / "pdf"

PDF_DIR.mkdir(parents=True, exist_ok=True)

assert NOTEBOOK_PATH.exists(), f"Notebook not found: {NOTEBOOK_PATH}"

```

```

PDF_PATH = PDF_DIR / f"{NOTEBOOK_PATH.stem}.pdf"

print("Notebook :", NOTEBOOK_PATH)
print("PDF      :", PDF_PATH)
print("\nExporting with nbconvert + XeLaTeX...")

result = subprocess.run(
    [
        sys.executable,
        "-m",
        "jupyter",
        "nbconvert",
        "--to",
        "pdf",
        "--output",
        NOTEBOOK_PATH.stem,
        "--output-dir",
        str(PDF_DIR),
        str(NOTEBOOK_PATH),
    ],
    capture_output=True,
    text=True,
)

if result.returncode != 0:
    print("\n--- stdout ---")
    print(result.stdout)

    print("\n--- stderr ---")
    print(result.stderr)

    raise RuntimeError("PDF EXPORT: FAILED")

assert PDF_PATH.exists(), f"PDF was not created: {PDF_PATH}"

print("\n" + "=" * 70)
print("PDF EXPORT: PASS")
print("=" * 70)
print("Saved:", PDF_PATH)

```

```

Notebook : d:\VGU-27\Edge AI Project\notebooks\current\00_prepare_dataset.ipynb
PDF      : d:\VGU-27\Edge AI Project\reports\pdf\00_prepare_dataset.pdf

```

Exporting with nbconvert + XeLaTeX...

```

=====
PDF EXPORT: PASS

```

=====
Saved: d:\VGU-27\Edge AI Project\reports\pdf\00_prepare_dataset.pdf